

TD Mémoire Partagée

Exercice 1 :

Créer un programme avec deux processus fils (fils1 et fils2) qui partagent une variable entière initialisée à zéro. Fils1 effectue 500000 opérations d'incrémentations de cette variable, tandis que fils2 effectue 5000 opérations de décrémentation.

Correction :

(Sans sémaphore)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/ipc.h>
5 #include <sys/shm.h>
6 #include <sys/wait.h>
7 #include <semaphore.h>
8 #include <fcntl.h> // Pour O_CREAT
9 #define N 500000 // Nombre d'opérations à effectuer
10 int main() {
11     int shmid;
12     int *shared_var;
13     shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
14     // Attacher la mémoire partagée
15     shared_var = (int*)shmat(shmid, NULL, 0);
16     // Initialiser la variable partagée à zéro
17     *shared_var = 0;
18     // Créer le premier processus fils (incrémentations)
19     pid_t pid1 = fork(); // Premier processus fils : incrémente la variable 500000 fois
20     if (pid1 == 0) {
21         for (int i = 0; i < N; i++) {
22             (*shared_var)++;
23         }
24         printf("Premier fils a terminé l'incrémentations.\n");
25         exit(0);
26     }
27     // Créer le deuxième processus fils (décrémentation)
28     pid_t pid2 = fork();
29     if (pid2 == 0) {
30         // Deuxième processus fils : décrémente la variable 500000 fois
31         for (int i = 0; i < N; i++) {
32             (*shared_var)--;
33         }
34         exit(0);
35     }
36     // Attendre que les deux processus fils terminent
37     wait(NULL);
38     wait(NULL);
39     // Afficher la valeur finale de la variable partagée
40     printf("La valeur finale de la variable partagée est: %d\n", *shared_var);
41     return 0;
42 }
```

```
aymen@aymen-virtual-machine:~/Desktop$ gcc exx1.c
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Premier fils a terminé l'incrémentations.
La valeur finale de la variable partagée est: 179996
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Premier fils a terminé l'incrémentations.
La valeur finale de la variable partagée est: -21220
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Premier fils a terminé l'incrémentations.
La valeur finale de la variable partagée est: 90682
aymen@aymen-virtual-machine:~/Desktop$
```

(Avec sémaphore)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/ipc.h>
5 #include <sys/shm.h>
6 #include <sys/wait.h>
7 #include <semaphore.h>
8 #include <fcntl.h> // Pour O_CREAT
9 #define N 5000 // Nombre d'opérations à effectuer*
10 int main() {
11     int shmid;
12     int *shared_var;
13     sem_t *sem; // Déclaration du sémaphore nommé
14     sem = sem_open("/mysem_test", O_CREAT, 0644, 1); // Création du sémaphore nommé
15     // Créer un segment de mémoire partagée
16     shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
17     // Attacher la mémoire partagée
18     shared_var = (int*)shmat(shmid, NULL, 0);
19     *shared_var = 0;
20     pid_t pid1 = fork();
21     if (pid1 == 0) {
22         for (int i = 0; i < N; i++) {
23             sem_wait(sem); // Prendre le sémaphore
24             (*shared_var)++; //variable partagée
25             sem_post(sem); // Libérer le sémaphore
26         }
27         exit(0);
28     }
29     pid_t pid2 = fork();
30     if (pid2 == 0) {
31         for (int i = 0; i < N; i++) {
32             sem_wait(sem); // Prendre le sémaphore
33             (*shared_var)--; //variable partagée
34             sem_post(sem); // Libérer le sémaphore
35         }
36         printf("Deuxième fils a terminé la décrémentation.\n");
37         exit(0);
38     }
39     wait(NULL);
40     wait(NULL);
41     printf("La valeur finale de la variable partagée est: %d\n", *shared_var);
42     return 0;
43 }
```

```
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Deuxième fils a terminé la décrémentation.
La valeur finale de la variable partagée est: 0
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Deuxième fils a terminé la décrémentation.
La valeur finale de la variable partagée est: 0
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Deuxième fils a terminé la décrémentation.
La valeur finale de la variable partagée est: 0
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Deuxième fils a terminé la décrémentation.
La valeur finale de la variable partagée est: 0
aymen@aymen-virtual-machine:~/Desktop$
```

Exercice 2:

Créer deux processus : Deux fils qui partagent un tableau de 50000 entiers. Le Fils 1 insère l'entier 1 dans le tableau et le fils insère l'élément 2.

- À la fin du programme, le Père vous affiche le « winner » (le processus qui a mis plus d'éléments) et le « loser » (le processus qui a mis moins d'éléments).

Correction :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/ipc.h>
5 #include <sys/shm.h>
6 #include <sys/wait.h>
7 #define ARRAY_SIZE 50000
8 int main() {
9     // Création de la mémoire partagée
10    key_t key = ftok("shmfile", 65);
11    int shmid = shmget(key, ARRAY_SIZE * 4, 0666 | IPC_CREAT);
12    int *shared_array = (int *)shmat(shmid, NULL, 0);
13    // Initialisation du tableau partagé
14    for (int i = 0; i < ARRAY_SIZE; i++) {
15        shared_array[i] = 0;
16    }
17    pid_t pid1, pid2;
18    pid1 = fork();
19    if (pid1 == 0) { // Fils 1
20        for (int i = 0; i < ARRAY_SIZE; i++) {
21            if (shared_array[i] == 0) { // Insérer 1 si la case est vide
22                shared_array[i] = 1;
23            }
24        }
25        shmdt(shared_array);
26        exit(0);
27    }
28    pid2 = fork();
29    if (pid2 == 0) { // Fils 2
30        for (int i = 0; i < ARRAY_SIZE; i++) {
31            if (shared_array[i] == 0) { // Insérer 2 si la case est vide
32                shared_array[i] = 2;
33            }
34        }
35        shmdt(shared_array);
36        exit(0);
37    }
38    // Attendre la fin des deux fils
39    wait(NULL);
40    wait(NULL);
41    // Calculer le nombre d'éléments insérés par chaque fils
42    int count1 = 0, count2 = 0;
```

```

43     for (int i = 0; i < ARRAY_SIZE; i++) {
44         if (shared_array[i] == 1) count1++;
45         if (shared_array[i] == 2) count2++;
46     }
47     // Déterminer le gagnant et le perdant
48     printf("Résultats :\n");
49     printf("Fils 1 (1) a inséré %d éléments.\n", count1);
50     printf("Fils 2 (2) a inséré %d éléments.\n", count2);
51     if (count1 > count2) {
52         printf("Winner: Fils 1\n");
53         printf("Loser: Fils 2\n");
54     } else if (count2 > count1) {
55         printf("Winner: Fils 2\n");
56         printf("Loser: Fils 1\n");
57     } else {
58         printf("Égalité entre Fils 1 et Fils 2.\n");
59     }
60     return 0;
61 }
62

```

```

aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Résultats :
Fils 1 (1) a inséré 16453 éléments.
Fils 2 (2) a inséré 33547 éléments.
Winner: Fils 2
Loser: Fils 1
aymen@aymen-virtual-machine:~/Desktop$ ./a.out
Résultats :
Fils 1 (1) a inséré 38398 éléments.
Fils 2 (2) a inséré 11602 éléments.
Winner: Fils 1
Loser: Fils 2
aymen@aymen-virtual-machine:~/Desktop$

```